UWE Bristol | University of the West of England

Presentation by

Majid Mumtaz

Week 8: Introduction to GUI with Python

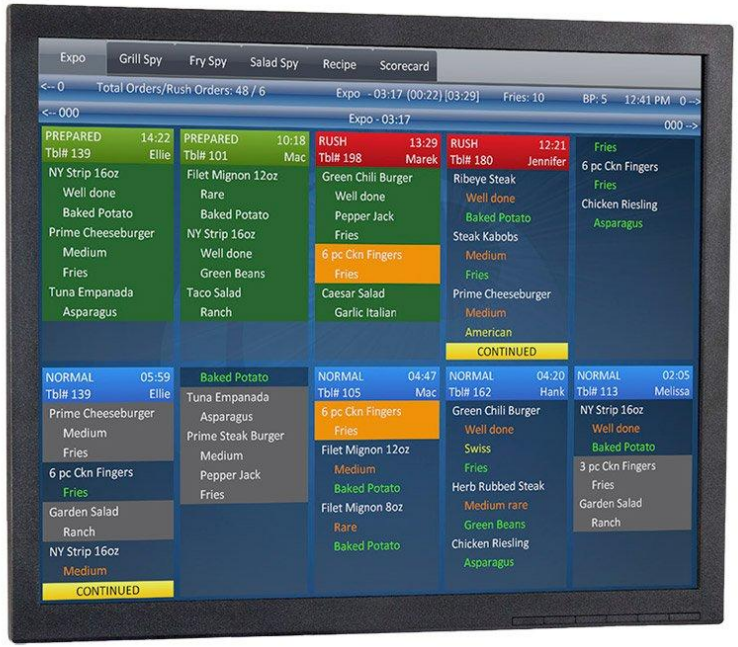
Week 8

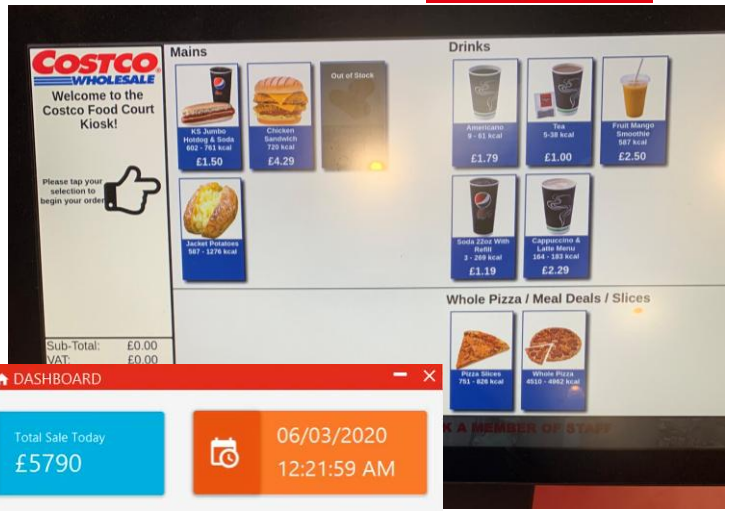
Recap from Last Lecture

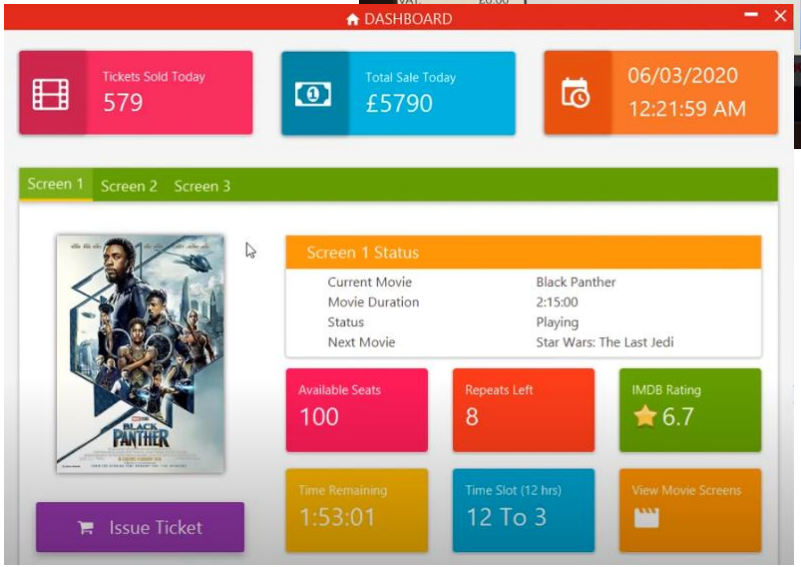
Continuous Integration
Continuous Delivery/Deployment
  CircleCI & GitHub Actions to create
  CI/CD Pipeline
    Source, Build, Test, Deploy
Configuration Management

Overview (Today's Plan)

- Graphical User Interface (GUI)
- Principles of User Interface Design
- Begin GUI programming with Python







@Image Source: https://tray.com/kitchen-display-system/ & Dashboard https://www.youtube.com/watch?v=CrQ3Fr306pM

Introduction

- Communication and interaction in between human and computers has become an important issue since the beginning of Computer Science discipline.

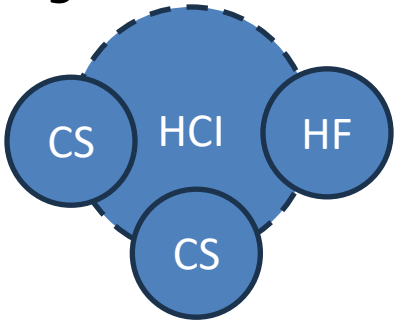
- It is about how to let human users interact with computers.

- The venues are keyboards, mouse/mise, JoyStick, microphones, cameras, touch screens, etc

- With new devices, this area of study is becoming more interesting … e.g., oculus headset …. Augmented Reality, Virtual Reality, Mixed Reality
- Generative AI tools to assist with UI/UX design

Human – Computer Interaction (HCI)

- **HCI** is a settled Computer Science discipline investigating how best human users can interact with the computing devices including modern handsets, robots, etc.
  - Computer Science/Design principles
  - Ergonomy, Cognitive Science
  - Human factors



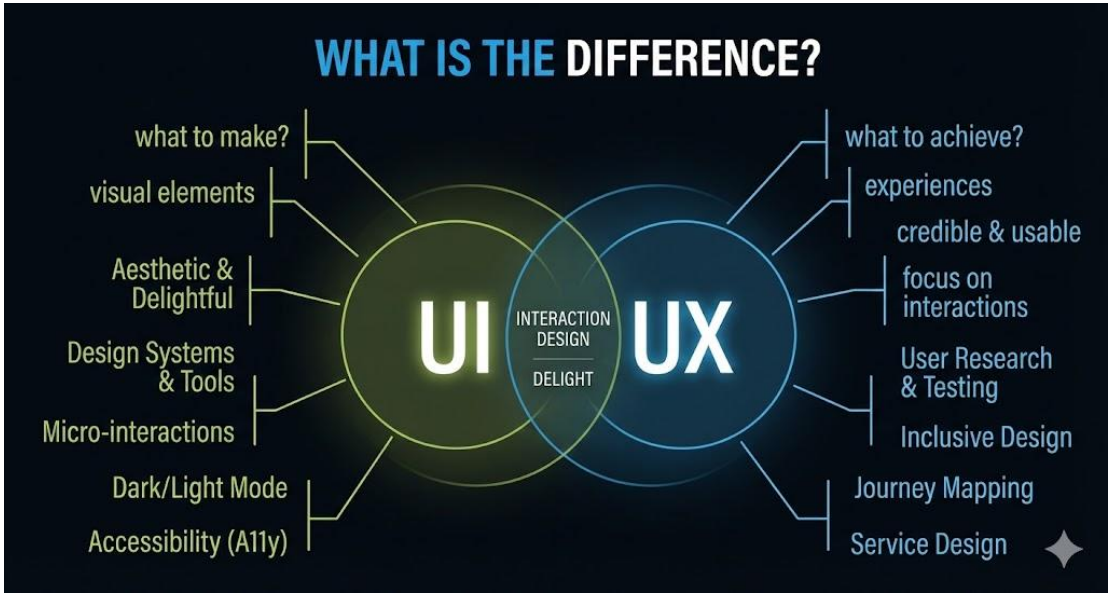
- We are mainly interested in **User Interfaces (UI)** as part of this module

Human-Computer Interaction (HCI):
https://www.interaction-design.org/literature/topics/human-computer-interaction?order_by=date

Encyclopedia of HCI (2nd Edition):
https://www.interaction-design.org/literature/book/the-encyclopedia-of-human-computer-interaction-2nd-ed
In particular chapter 42 – Design for all.

Human – Computer Interaction (HCI)

- **User eXperience and User Interface (UX/UI)**

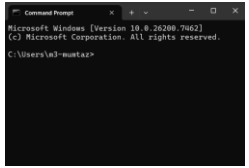


@source: https://aekpani.net/the-difference-between-ux-and-ui-design/ (Nanobnana edited)

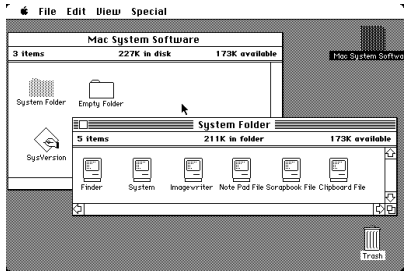
Read: UX vs UI: What's the difference?
https://www.interaction-design.org/literature/article/ux-vs-ui-what-s-the-difference

GUI History

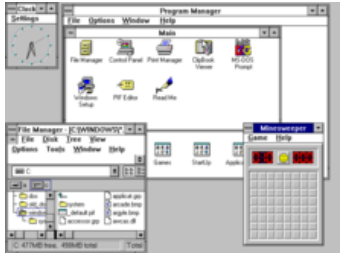
- Evolution in Windows
  - MS DOS based systems
  - Windows 95 was the GUI-based OS
- After 2000 to Date
  - 3D desktops
  - Mobile handsets
  - Interactive Touch Tables
  - VR headsets
- Others:
  - Voice controlled Siri, Alexa..
  - Gesture based interfaces
  - Agentic AI?



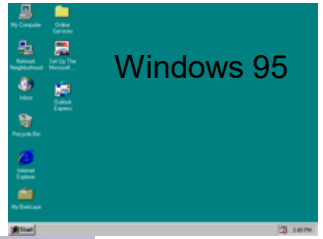
CMD (Terminal)



Macintosh Desktop (1984)



Windows 3.11 1993

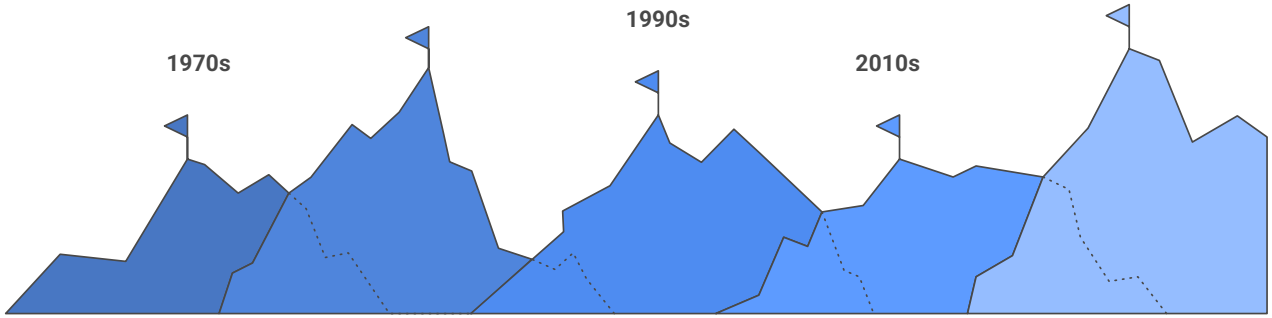


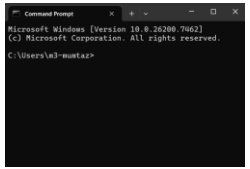
Touch Table

Foldable Mobile

VR Headsets

Evolution of GUI (1970 - 2026)

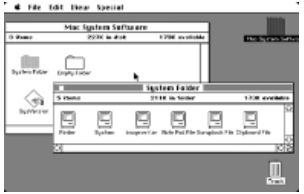




CMD (Terminal)

WIMP (Xerox)

Xerox Alto introduces windows, icons, menus, and pointers



Macintosh Desktop (1984)

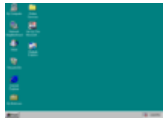
Desktop Metaphor

Apple Macintosh popularizes folders, trash, and icons on a desktop

Windows 3.11
1993

Task-Based

Windows 95 introduces Start Menu and Taskbar for task management



Windows 95

Windows 8

Flat Design

iOS 7 and Android Material Design embrace minimalism and grid layouts

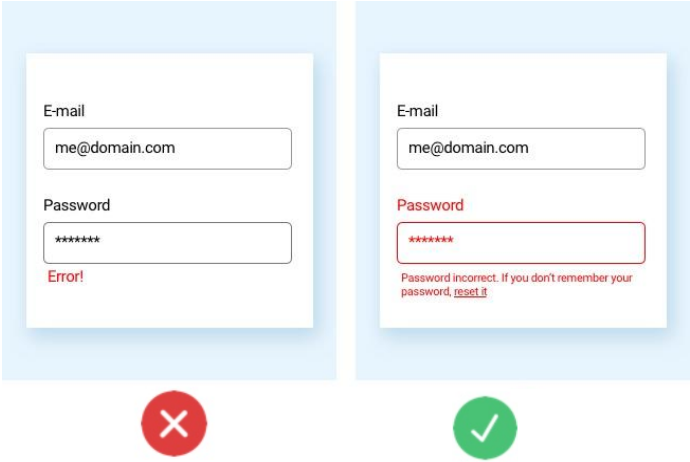
Agentic UI

AI-powered interfaces adapt and generate content based on voice and gestures

Why Needs for UI design principles

- Software systems are purchased but remained not used due to complex UIs.
- Lack usability and user interfaces.

  - A system will be judged by its users by its interfaces rather than its functionalities since the functionalities are usually successfully developed.
  - A well-designed interface can help use the systems smoothly while a poorly designed interface can cause users bad experience and can make a catastrophic errors
  - Poorly designed user interfaces can cause many software systems never to be used



Pointless inconsistency in your UI elements
Little distinction between primary and secondary buttons
Lack of text hierarchy
Unaligned elements
Low contrast
Confusing forms
Etc.

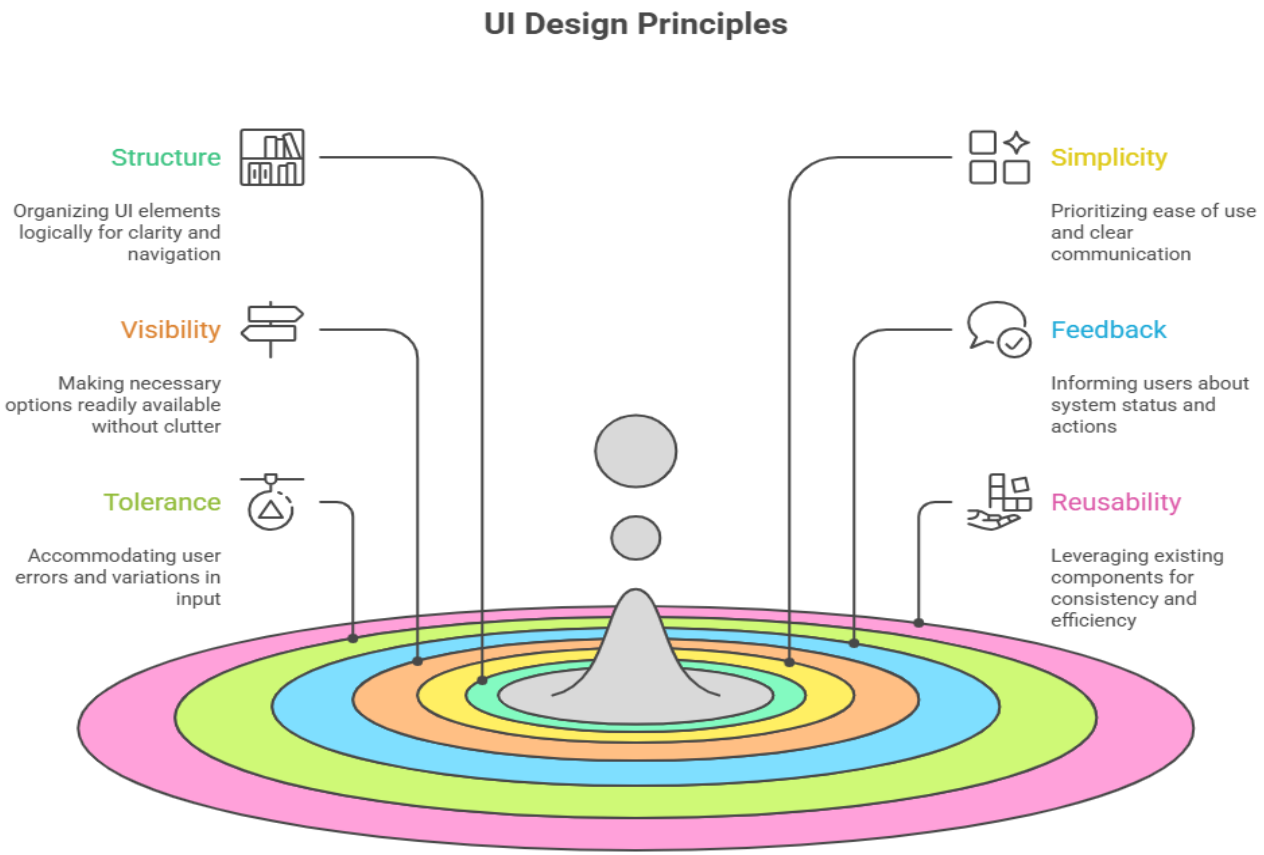
Image source and interesting 8 min read: https://careerfoundry.com/en/blog/ui-design/common-ui-design-mistakes/

UI Design principles

Principle	*Description*
User familiarity	**The interface should use terms and concepts which are drawn from the experience of the people who will make most use of the system.**
Consistency	**The interface should be consistent in that, wherever possible, comparable operations should be activated in the same way.**
Minimal surprise	**Users should never be surprised by the behaviour of a system.**
Recoverability	**The interface should include mechanisms to allow users to recover from errors.**
User guidance	**The interface should provide meaningful feedback when errors occur and provide context-sensitive user help facilities.**
User diversity	**The interface should provide appropriate interaction facilities for different types of system user.**

Software Engineering by Ian Sommerville (8th Edition), Chapter 16, pg. 364

UI Design Principles by Lucy Lockwood



Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design, Released April 1999 Publisher(s): Addison-Wesley Professional

UI Design principles

- Structure
  - This principle is concerned with the overall architecture of the interface design.
  - The design should organize the user interface purposefully,
    - in meaningful and useful ways based on clear, consistent models that are apparent and recognizable to users,
    - putting related things together and separating unrelated things, differentiating dissimilar things
    - making similar things resemble one another.

- Simplicity
  - The design should make simple, common tasks easy, communicating clearly and simply in the user's own language,
  - and providing good shortcuts that are meaningfully related to longer procedures.

- Visibility
  - The design should make all needed options and materials for a given task visible without distracting the user with extraneous or redundant information.
  - Good designs don't overwhelm users with alternatives or confuse with unneeded information.

By Lucy Lockwood

Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design, Released April 1999
Publisher(s): Addison-Wesley Professional

UI Design principles

UWE Bristol University of the West of England

- Feedback
  - The design should keep users informed of:
    - actions or interpretations, changes of state or condition,
      - errors or exceptions that are relevant and of interest to the user through clear, concise, and unambiguous language familiar to users.

- Tolerance
  - The design should be flexible and tolerant, reducing the cost of mistakes and misuse by allowing undoing and redoing,
  - It should also be preventing errors wherever possible by tolerating varied inputs and sequences and by interpreting all reasonable actions reasonable.

- Reusability

  The design should reuse internal and external components and behaviours, maintaining consistency with purpose rather than merely arbitrary consistency, thus reducing the need for users to rethink and remember.

Did you notice something unusual on this slide?

By Lucy Lockwood

GUI Programming

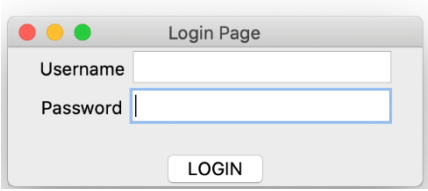
- **Graphical UI** is a special form of UI that lets users interact via graphics, more attractive forms and elements.

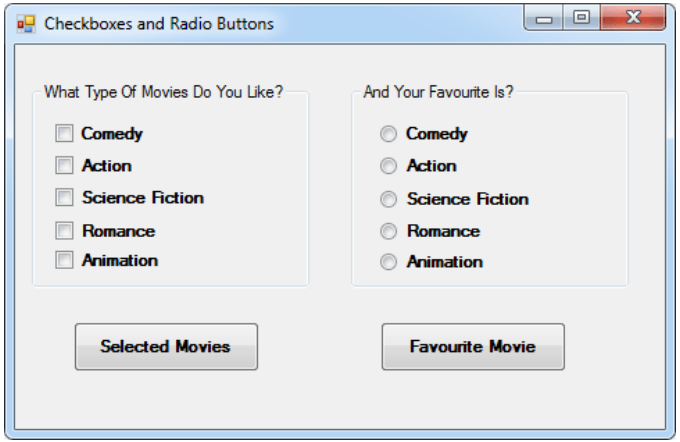
- **GUI programming** is a sub-field of computer programming and software development, and most high-level programming languages have paid attention to it.

GUI components

- GUI programming in many programming languages is an implementation of OOP, e.g., Python, Java

- There are many standard and key elements commonly used for this purpose:
  - Windows, panels, frames, canvas
  - Buttons, radio-buttons, check boxes
  - Text-fields and areas
  - Information, error dialog boxes





GUI programming with Python

Python Framework	Notes
PyQt6	PyQt6 is **fully cross-platform**. By leveraging the power of PyQt6, developers can build applications for platforms like Windows, Mac, Linux, iOS, Android, and more.
Tkinter	Standard Python library, comes with Python, easy to use
Kivy	Fully cross-platform. OpenGL ES accelerated framework for creating new user interfaces. Kivy have a native multitouch support for every platform, mouse simulator, widgets multitouch ready, integrated virtual keyboard, rst renderer
PyGTK	Based on the GTK toolkit, known for its flexibility

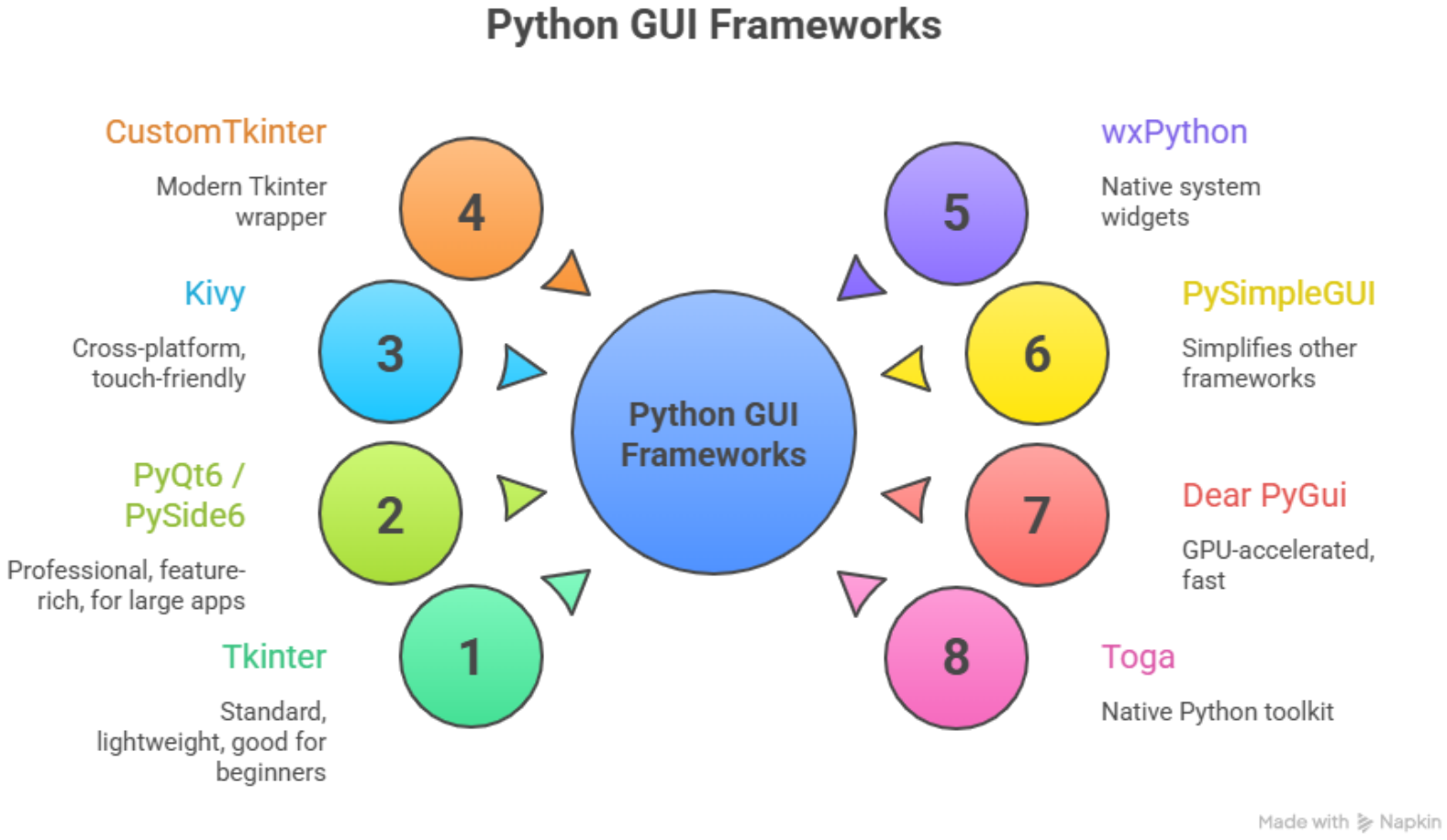
GUI programming with Python

Python Framework	Notes
wxPython	It is cross platform GUI API. It is a blend of the WxWidgets (formerly WxWindows) class library for C++ and Python. It's a possible alternative to Tkinter with a cross-platform compatibility.
PySimpleGUI	Wraps tkinter, Qt (pyside2), wxPython and Remi (for browser support) in a non-OOP API. Build custom GUI layouts in minutes in a few lines of code. Easy enough for beginners, powerful enough for advanced users. Capable of complex & extensive multi-window applications as well as simple ones.
PyForms	Its cross-platform capabilities make it easier for the developers to create applications for multiple platforms with little changes to the code, eliminating unnecessary slowdowns. Styling with CSS is also possible

Others especially if web compatibility is needed …. PyWebView. Eal, CEF Python

Review

GUI programming with Python



GUI programming with Tkinter

Python
tkinter
Tcl/Tk
Operating System
Hardware

To create a Tkinter app:

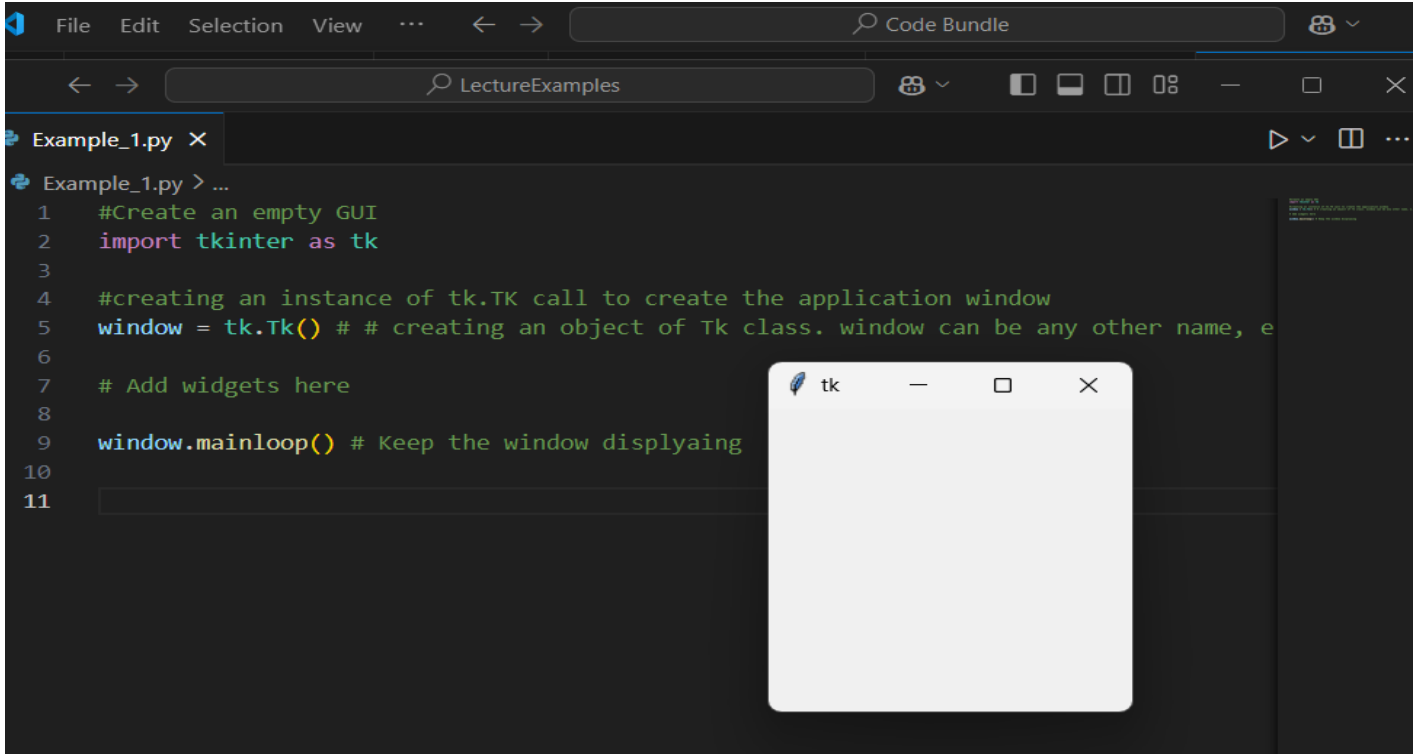
Tea – Kay – Inter

1. Importing the module – tkinter
2. Create the main window (container)
3. Add any number of widgets to the main window
4. Apply the event Trigger on the widgets.

```
import tkinter
m = tkinter.Tk()
'''
widgets are added here
'''
m.mainloop()
```

Official Documentation: https://docs.python.org/3/library/tkinter.html

GUI programming with Tkinter



*Example_1.py

Code & Output

*Official Documentation: https://docs.python.org/3/library/tkinter.html

Widgets

- GUI items (aka widgets in Tkinter) e.g.,

Widget Type	Purpose
Input	Entry, Text, Spinbox, Scale, Combobox (ttk)
Selection	Checkbutton, Radiobutton, Listbox, Treeview (ttk)
Container	Frame, LabelFrame, PanedWindow, Notebook (ttk), Toplevel
Display	Label, Message, Progressbar (ttk), Canvas, Separator (ttk)
Action	Button, Menubutton

Label
Text
Frame
Scrollbar
ScrolledText
Separator
Checkbox
Radio Button
Combobox
Listbox

LabelFrame
Notebook
Treeview

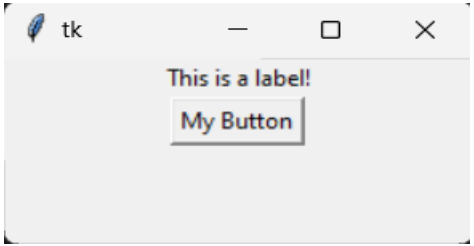
Many others…
e.g., Slider, Spinbox, Sizegrip, Progressbar, Canvas, Cursors

GUI programming with Tkinter

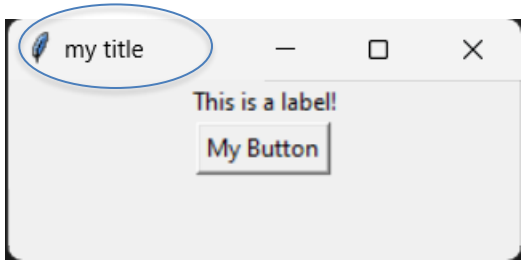
1. Importing the module – tkinter
2. Create the main window (container)
3. Add any number of widgets to the main window
4. Apply the event Trigger on the widgets.

```
#Create a GUI with Button and Label
import tkinter as tk

window = tk.Tk()
window.title("my title")
label = tk.Label(text="This is a label!");
label.pack()

button = tk.Button(master=window, text="My Button")
button.pack()

window.mainloop()
```

Example_1b.py



Default Title



Window.title("my title")

OOP and TKinter

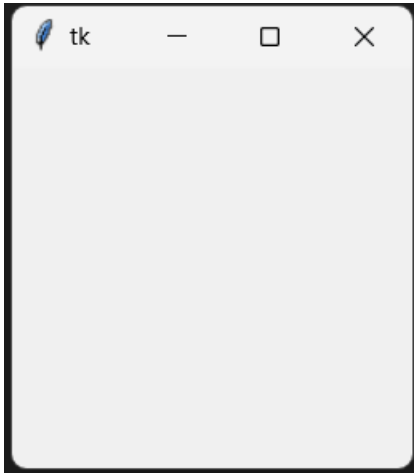
When program/code is becoming more complex, you should use OOP to organise your code

```
from tkinter import *
w = Tk()
w.mainloop()
```

```
import tkinter as tk

class App(tk.Tk):
    def __init__(self):
        super().__init__()

if __name__ == '__main__':
    app = App()
    app.mainloop()
```

Example_1_with_OOP.py

Output



GUI programming with Tkinter

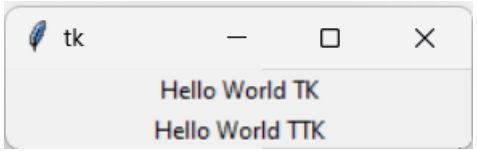
UWE Bristol University of the West of England

What are **Tkinter Theme** widgets? :

1. *from tkinter import ttk*
2. ttk is a TK themed widgets introduced in 2007 and replace many but not all TK widgets
3. Use *ttk* where possible and available
   a. Adopts native look and feel of the platform on which app runs
   b. Keeps behaviour code separate from the appearance
4. Some new ttk widgets e.g., *Progeressbar, Treeview, Separator*, etc.

```python
import tkinter as tk
from tkinter import ttk

# creating an instance of tk.TK call to create the application window
root = tk.Tk() # root can be any other name. root is by convension

# place a tk label on the root window
tk_message = tk.Label(root, text="Hello, World TK!")
tk_message.pack()

# place a ttk label on the root window
ttk_message = ttk.Label(root, text="Hello, World TTK!")
ttk_message.pack()

# keep the window displaying
root.mainloop()
```

Example_1c.py



Output of tk and ttk label

GUI programming with Tkinter

Three common ways to set the options for ttk widgets:

1. *Using keyword arguments when creating a widget*
2. *Using dictionary index after widget creation*
3. *Using config() method with keyword attributes*

Example_1d.py

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
ttk.Label(root, text='Hi, there').pack()
root.mainloop()
```

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
mylabel = ttk.Label(root)
Mylabel['text'] = 'Hi, there'
Mylabel.pack()
root.mainloop()
```

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
mylabel = ttk.Label(root)
Mylabel.config(text = 'Hi, there')
Mylabel.pack()
root.mainloop()
```

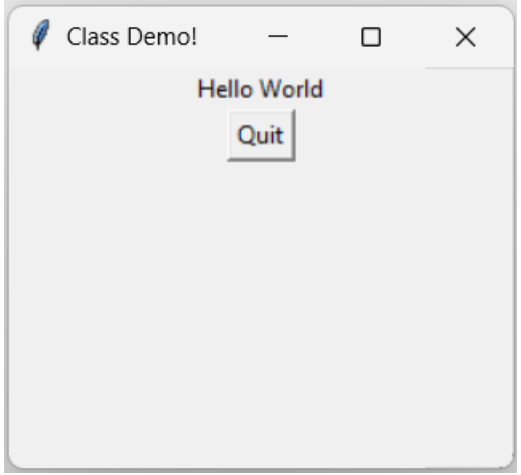
GUI with Tkinter

- Widget's geometry is managed with **pack()** object
- Widgets classes offer a number of parameters to use the elements in different form and purposes.
- command is used to take action once pressed

Example_1f.py > ...

```python
#Create an empty GUI
import tkinter as tk
import tkinter.messagebox as tkm

window = tk.Tk()

window.title('Class Demo!')
window.geometry('200x200')

label = tk.Label(text = 'Hello World')
label.pack()

button = tk.Button(window, command=window.destroy, text = 'Quit')
button.pack()


window.mainloop()
```

Example_1f.py



GUI programming with Tkinter

UWE Bristol University of the West of England

- A very important component of Tkinter is the geometry manager, which helps shape the widgets within a container.

- Three classes:
  - **pack()** method organises the widgets in blocks before placing in the parent widget.

  - **grid()** method organises the widgets in grid (table-like structure) before placing in the parent widget.

  - **place()** method organises the widgets by placing them on specific positions directed by the programmer.

GUI programming with Tkinter

UWE Bristol University of the West of England

pack()

It is one of the easiest to use, compared to the other 2 geometry managers.

The syntax is

```
widget.pack(options,…)
```

The options are:

- **fill**: This option will determine whether the widgets can increase or grow in size or not. By default, it is NONE. If we want to fill vertically, then it is Y. If horizontally, then it is X. If required both horizontally or vertically, then BOTH.
- **expand**: This option will expand the widget to fill any space when set to true or 1. When the window is resized, the widget will expand.
- **side**: This option will decide the widget alignment, that is, against which side of the parent widget packs. By default, it is TOP. The others are BOTTOM, LEFT, or RIGHT.

GUI with Tkinter – pack()

```python
root1 = tk.Tk()
root1.geometry("350x200+900+600")

# box 1
box1 = tk.Label(root1, text="Box 1", bg="green", fg="white")
box1.pack(ipadx=10, ipady=10, fill=tk.X)

# box 2
box2 = tk.Label(root1, text="Box 2", bg="red", fg="white")
box2.pack(ipadx=10, ipady=10, fill=tk.X)

# box 3
box3 = tk.Label(root1, text="Box 3", bg="blue", fg="white", )
box3.pack(ipadx=10, ipady=10, fill=tk.X)

# box 4
box4 = tk.Label(root1, text="Box 4", bg="cyan", fg="black", )
box4.pack(ipadx=10, ipady=10, side=tk.LEFT, fill=tk.BOTH, expand=True)

# box 5
box5 = tk.Label(root1, text="Box 5", bg="purple", fg="white", )
box5.pack(ipadx=10, ipady=10, side=tk.LEFT, fill=tk.BOTH, expand=True)

# keep the window displaying
root1.mainloop()
```

pack() is suitable for placing widgets in a top-down layout and/or side-by-side layout.

pack(fill, expand, side, anchor, ipadx, ipady, padx, and pady)

Example_1i.py

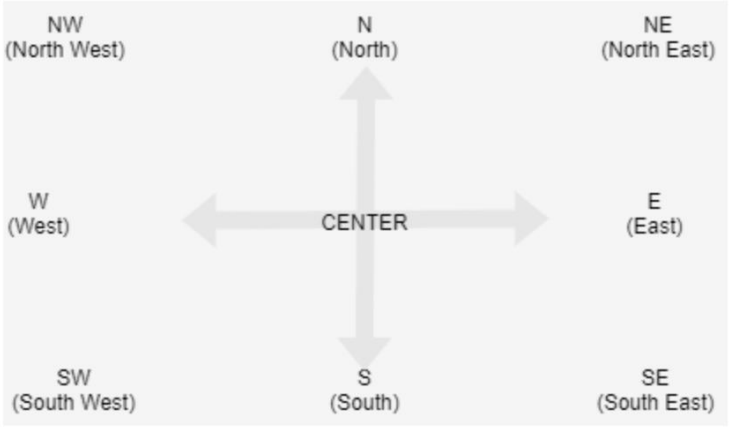
Source:
https://www.pythontutorial.net/tkinter/tkinter-pack/

GUI with Tkinter

("X-width x Y-width + position x-axis + position y-axis)

pack() - Anchor option

```
root1 = tk.Tk()
root1.geometry("350x200+900+600")

# box 1
box1 = tk.Label(root1, text="Box 1", bg="green", fg="white")
box1.pack(ipadx=10, ipady=10, fill=tk.BOTH, expand=True)

# box 2
box2 = tk.Label(root1, text="Box 2", bg="red", fg="white")
box2.pack(ipadx=10, ipady=10, anchor=tk.W)

# box 3
box3 = tk.Label(root1, text="Box 3", bg="blue", fg="white")
box3.pack(ipadx=10, ipady=10, anchor=tk.E)


# keep the window displaying
root1.mainloop()

# keep the window displaying
root.mainloop()
```

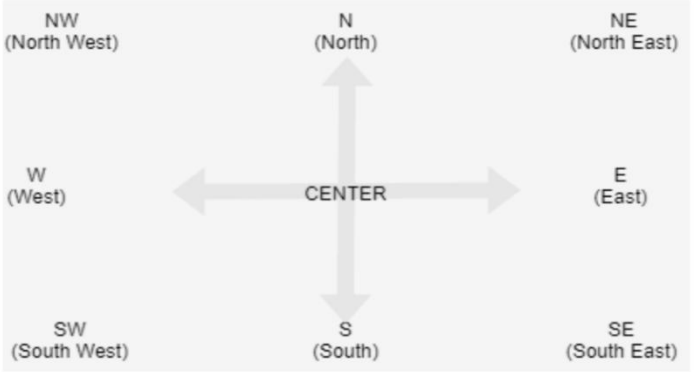


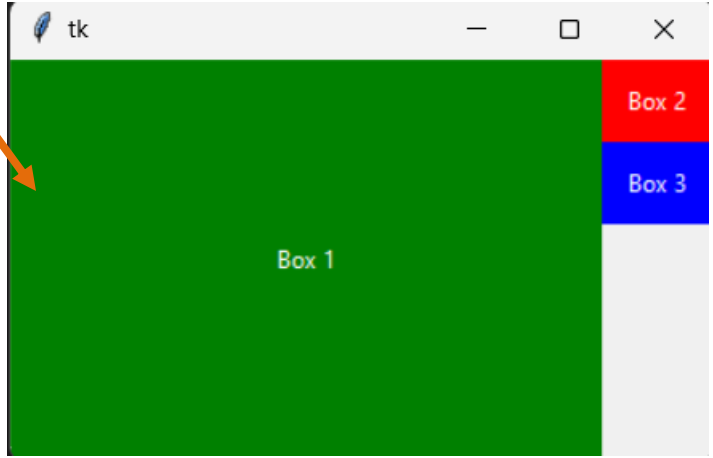
Example_1j.py

GUI with Tkinter

pack() - side option for alignment of a widget: top (default), left, right, bottom

```
root1 = tk.Tk()
root1.geometry("350x200+900+600")

# box 1
box1 = tk.Label(root1, text="Box 1", bg="green", fg="white")
box1.pack(ipadx=10, ipady=10, fill=tk.BOTH, expand=True, side=tk.LEFT)

# box 2
box2 = tk.Label(root1, text="Box 2", bg="red", fg="white")
box2.pack(ipadx=10, ipady=10, anchor=tk.W)

# box 3
box3 = tk.Label(root1, text="Box 3", bg="blue", fg="white", )
box3.pack(ipadx=10, ipady=10, anchor=tk.E)


# keep the window displaying
root1.mainloop()
```



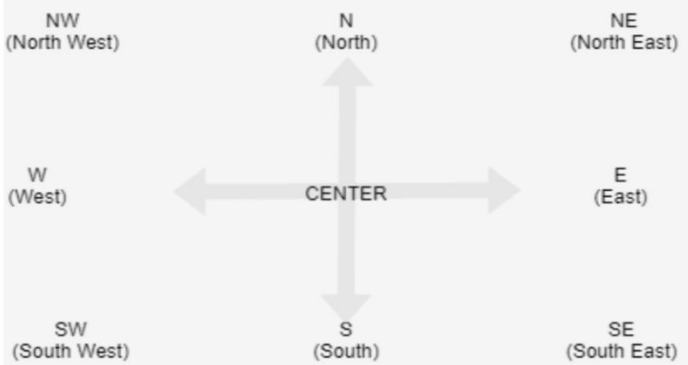


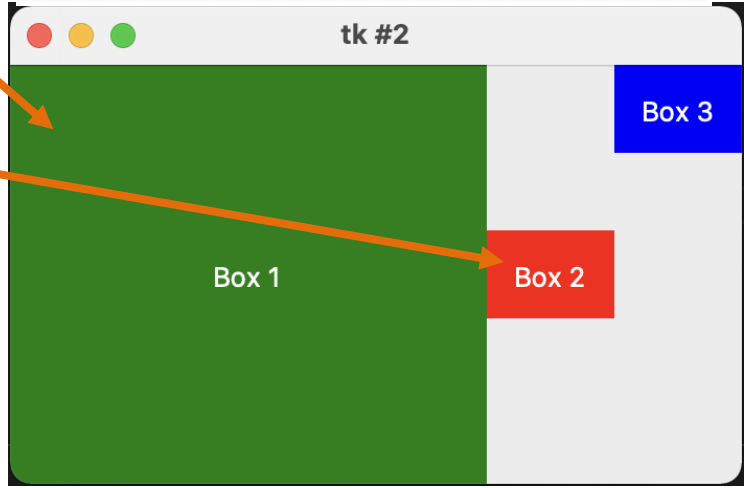
Example_1j.py

GUI with Tkinter

pack() - side option for alignment of a widget: top (default), left, right, bottom

```
root1 = tk.Tk()
root1.geometry("350x200+900+600")

# box 1
box1 = tk.Label(root1, text="Box 1", bg="green", fg="white")
box1.pack(ipadx=10, ipady=10, fill=tk.BOTH, expand=True, side=tk.LEFT)

# box 2
box2 = tk.Label(root1, text="Box 2", bg="red", fg="white")
box2.pack(ipadx=10, ipady=10, anchor=tk.W, side=tk.LEFT)

# box 3
box3 = tk.Label(root1, text="Box 3", bg="blue", fg="white", )
box3.pack(ipadx=10, ipady=10, anchor=tk.E)


# keep the window displaying
root1.mainloop()
```



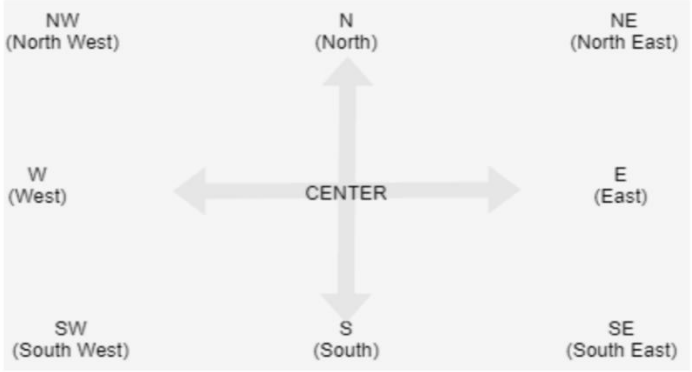


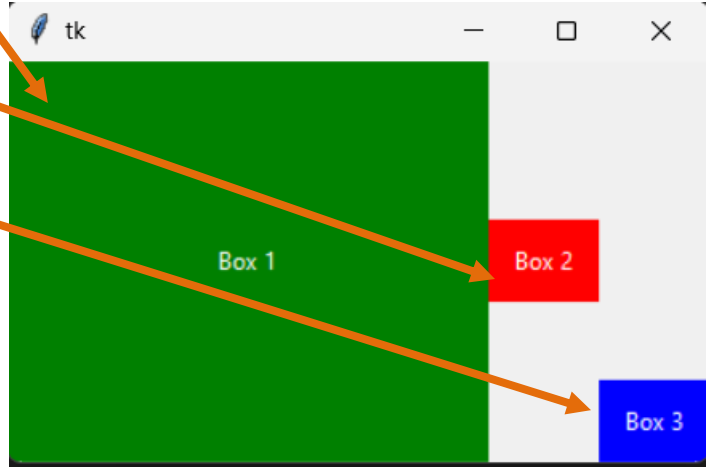
Example_1j.py

GUI with Tkinter

pack() - side option for alignment of a widget: top (default), left, right, bottom

```python
root1 = tk.Tk()
root1.geometry("350x200+900+600")

# box 1
box1 = tk.Label(root1, text="Box 1", bg="green", fg="white")
box1.pack(ipadx=10, ipady=10, fill=tk.BOTH, expand=True, side=tk.LEFT)

# box 2
box2 = tk.Label(root1, text="Box 2", bg="red", fg="white")
box2.pack(ipadx=10, ipady=10, anchor=tk.W, side=tk.LEFT)

# box 3
box3 = tk.Label(root1, text="Box 3", bg="blue", fg="white", )
box3.pack(ipadx=10, ipady=10, anchor=tk.E, side=tk.BOTTOM)


# keep the window displaying
root1.mainloop()
```



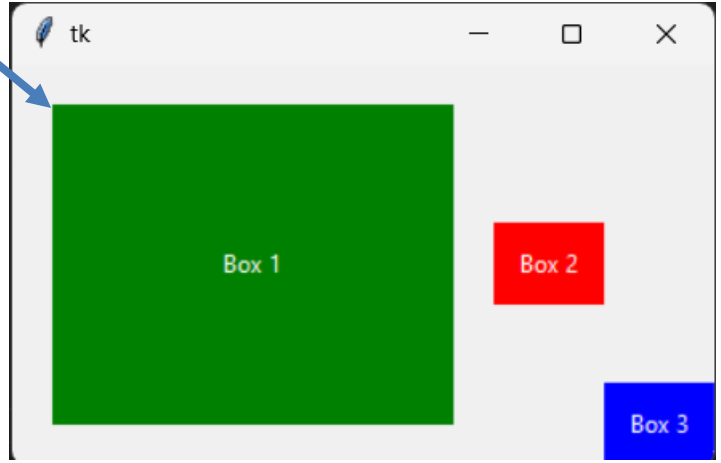


Example_1j.py

GUI with Tkinter

pack() – padx, pady

```
# box 1
box1 = tk.Label(root1, text="Box 1", bg="green", fg="white")
box1.pack(ipadx=10, ipady=10, padx=20, pady=20, fill=tk.BOTH, expand=True, side=tk.LEFT)
```



Example_1j.py

GUI with Tkinter - grid()

Grid geometry manager uses rows and columns to arrange widgets.

grid(column, row, sticky, padx, pady, ipadx, ipady)

In following example, widget's geometry is managed with grid() object

Checkbutton() is used to hold some values.

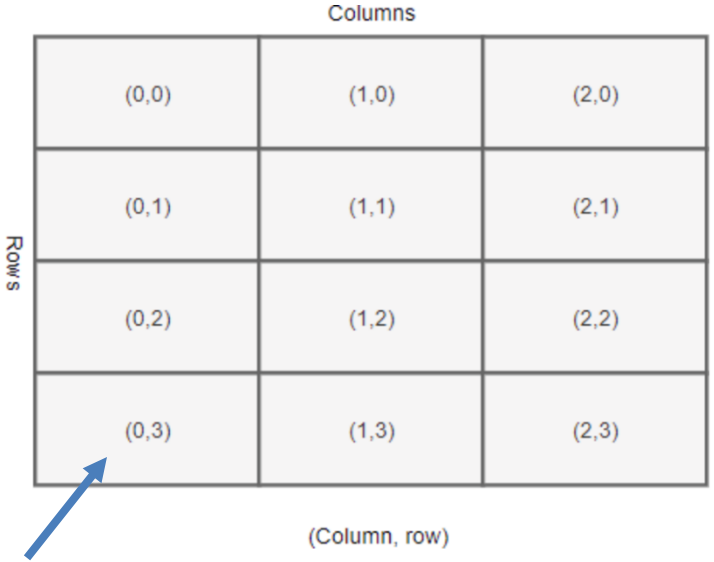
This can be extended with further details. ….

ASD > Examples > Widgets > checkboxwidget.py > ...

```python
from tkinter import *

root = Tk()
root.title('Checkbox example')
root.geometry('200x70')

var1 = IntVar()
Checkbutton(root, text='Python', variable=var1).grid(row=0, sticky=W)

var2 = IntVar()
Checkbutton(root, text='Java', variable=var2).grid(row=1, sticky=W)

mainloop()
```

Example_2a.py

Grid geometry manager:
https://www.pythontutorial.net/tkinter/tkinter-grid/



Cell – to place a widget (one widget)..
To place multiple widgets in one cell, use Frame

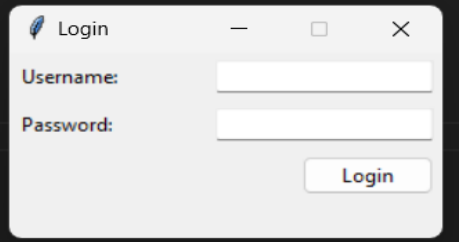
Width and height of a columns/rows depend on the w/h of the widget contained with col/rows

Column and **Row** can span e.g., columnspan=2.
Padding padx/y, ipadx/y can be specified
Sticky lets put the widget on a specific side if the cell is larger than widget. Default is CENTER.
Other options are N, W, S, E, NW, NE, SE, SW, NS, EW

GUI with Tkinter – Login GUI Example

Example_3a_loginexamplegrid.py > ...

```python
import tkinter as tk
from tkinter import ttk

def login():
    print("User name : " + username.get() + "\nPassword : " + password.get())

root = tk.Tk()
root.title('Login')
root.geometry("250x120")
root.resizable(0,0)

root.columnconfigure(0, weight=1)   #1st column width is 1
root.columnconfigure(1, weight=3)   #2nd columns is almost 3 times width to 1st column

username = tk.StringVar()
password = tk.StringVar()

username_label = ttk.Label(root,text='Username:').grid(column=0, row=0, sticky=tk.W, padx=5, pady=5)
username_entry = ttk.Entry(root, textvariable=username).grid(column=1, row=0, sticky=tk.E, padx=5, pady=5)

password_label = ttk.Label(root, text='Password:').grid(column=0, row=1, sticky=tk.W, padx=5, pady=5)
password_entry = ttk.Entry(root, textvariable=password, show="*").grid(column=1, row=1, sticky=tk.E, padx=5, pady=5)

login_button = ttk.Button(root, text='Login', command=login).grid(column=1, row=3, sticky=tk.E, padx=5, pady=5)

root.mainloop()
```

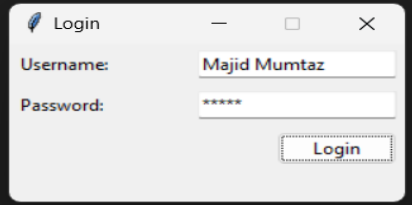


Example_3a_login.py

GUI with Tkinter – Login Example - OOP

Example_3c_loginexamplegrid_oop.py > ...

```python
import tkinter as tk
from tkinter import ttk

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.geometry("250x120")
        self.title('Login')
        self.resizable(0, 0)
        self.columnconfigure(0, weight=1)
        self.columnconfigure(1, weight=3)

        self.create_widgets()

    def create_widgets(self):
        username = tk.StringVar()
        password = tk.StringVar()
        username_label = ttk.Label(self, text="Username:").grid(column=0, row=0, sticky=tk.W, padx=5, pady=5)
        username_entry = ttk.Entry(self, textvariable=username).grid(column=1, row=0, sticky=tk.E, padx=5, pady=5)
        password_label = ttk.Label(self, text="Password:").grid(column=0, row=1, sticky=tk.W, padx=5, pady=5)
        password_entry = ttk.Entry(self, textvariable=password, show="*").grid(column=1, row=1, sticky=tk.E, padx=5, pady=
        login_button = ttk.Button(self, text="Login", command=lambda : self.login(username, password)).grid(column=1, row=

    def login(self, username, password):
        print("User name : " + username.get() + "\nPassword : " + password.get())

if __name__ == "__main__":
    app = App()
    app.mainloop()
```



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

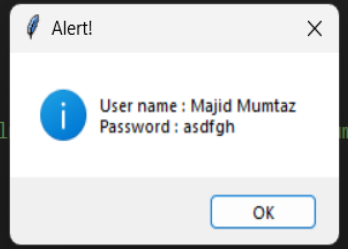
```
User name : Majid Mumtaz
Password : hello
```

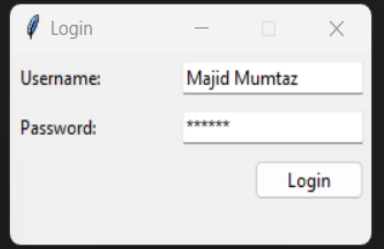
Example_3c_loginexamplegrid_oop.py

GUI with Tkinter – Login GUI Example - OOP

Example_3b_loginexamplegrid_V1.py > ...

```python

import tkinter as tk
from tkinter import ttk
import tkinter.messagebox as tkm

def login():
    #print("User name : " + username.get() + "\nPassword : " + password.get())
    tkm.showinfo("Alert!", "User name : " + username.get() + "\nPassword : " + password.get())

root = tk.Tk()
root.title('Login')
root.geometry("250x120")
root.resizable(0,0)

root.columnconfigure(0, weight=1)   #1st column width
root.columnconfigure(1, weight=3)   #2nd columns is al                    mn

username = tk.StringVar()
password = tk.StringVar()

username_label = ttk.Label(root,text='Username:').grid(column=0, row=0, sticky=tk.W, padx=5, pady=5)
username_entry = ttk.Entry(root, textvariable=username).grid(column=1, row=0, sticky=tk.E, padx=5, pady=5)

password_label = ttk.Label(root, text='Password:').grid(column=0, row=1, sticky=tk.W, padx=5, pady=5)
password_entry = ttk.Entry(root, textvariable=password, show="*").grid(column=1, row=1, sticky=tk.E, padx=5, pady=5)

login_button = ttk.Button(root, text='Login', command=login).grid(column=1, row=3, sticky=tk.E, padx=5, pady=5)

root.mainloop()
```





Example_3b_loginexamplegrid_V1.py

#Adapted from Source: https://www.pythontutorial.net/tkinter/tkinter-pack/

GUI with Tkinter - place()

UWE Bristol University of the West of England

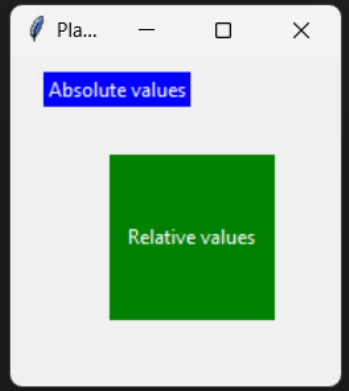
- Rarely used because changing on widget place will require adjusting all other widgets…
- Useful when end-user would like to place one/more specific widget(s) on specific place on the application window

- widget.place(**options) – *x, y, relx, rely, width, height, relwidth, relheight, anchor*
- *x,y are for absolute positions and relx,rely are for relative positions*
- *width,height for absolute values and relwidth,relheight* for *relative values (floating point b/w 0.0 and 1.0* - represent a fraction of the w/h of the container*)*
- *anchor – N, E, S, W, NW, SE, or SW – default NW*

GUI with Tkinter - place()

Example_4a_placegeomertry.py > ...

```python
import tkinter as tk

root = tk.Tk()
root.title('Place Geometry Manager - Example')

label1 = tk.Label(root, text='Absolute values', bg='blue', fg='white').place(x=20, y=10)
label2 = tk.Label(root, text='Relative values', bg='green', fg='white').place(relx=0.8, rely=0.8, relwidth=0.5, relheight=0.5, anchor='se')

root.mainloop()
```



Example_4a_placegeomertry.py

Read: https://www.pythontutorial.net/tkinter/tkinter-place/

GUI with Tkinter

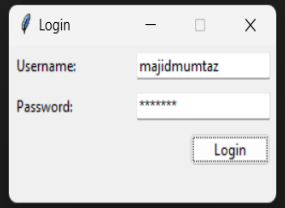
```python
from tkinter import *

def validate():
    print("Username entered :", user.get())

#window
w = Tk()
w.geometry('300x100')
w.title('Info Form')

#user label and text entry box
userLabel = Label(w, text="User Name").grid(row=0, column=0)
user = StringVar()
userEntry = Entry(w, textvariable=user).grid(row=0, column=1)

loginButton = Button(w, text="Login", command=validate).grid(row=4, column=0)

w.mainloop()
```

Pay attention to user-defined function of validate(), which is used to take an action once the button is pressed.
The entered string/text is picked up and printed out on the console.

GUI with Tkinter – Login GUI Example

Example_3c_loginexamplegrid_oop.py

```python
import tkinter as tk
from tkinter import ttk

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.geometry("250x120")
        self.title('Login')
        self.resizable(0, 0)
        self.columnconfigure(0, weight=1)
        self.columnconfigure(1, weight=3)

        self.create_widgets()

    def create_widgets(self):
        username = tk.StringVar()
        password = tk.StringVar()
        username_label = ttk.Label(self, text="Username:").grid(column=0, row=0, sticky=tk.W, padx=5, pady=5)
        username_entry = ttk.Entry(self, textvariable=username).grid(column=1, row=0, sticky=tk.E, padx=5, pady=5)
        password_label = ttk.Label(self, text="Password:").grid(column=0, row=1, sticky=tk.W, padx=5, pady=5)
        password_entry = ttk.Entry(self, textvariable=password, show="*").grid(column=1, row=1, sticky=tk.E, padx=5, pady=5)
        login_button = ttk.Button(self, text="Login", command=lambda : self.login(username, password)).grid(column=1, row=3, sticky=tk.E, padx=5, pa

    def login(self, username, password):
        print("User name : " + username.get() + "\nPassword : " + password.get())

if __name__ == "__main__":
    app = App()
    app.mainloop()
```



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL CONSOLE COMMENTS

```
/Example_3c_loginexamplegrid_oop.py"
User name : majidmumtaz
Password : test123
```

Pay attention to command field for Button

#Adapted from Source: https://www.pythontutorial.net/tkinter/tkinter-pack/

GUI with Tkinter – Login Example - OOP

UWE Bristol University of the West of England

Requirements:

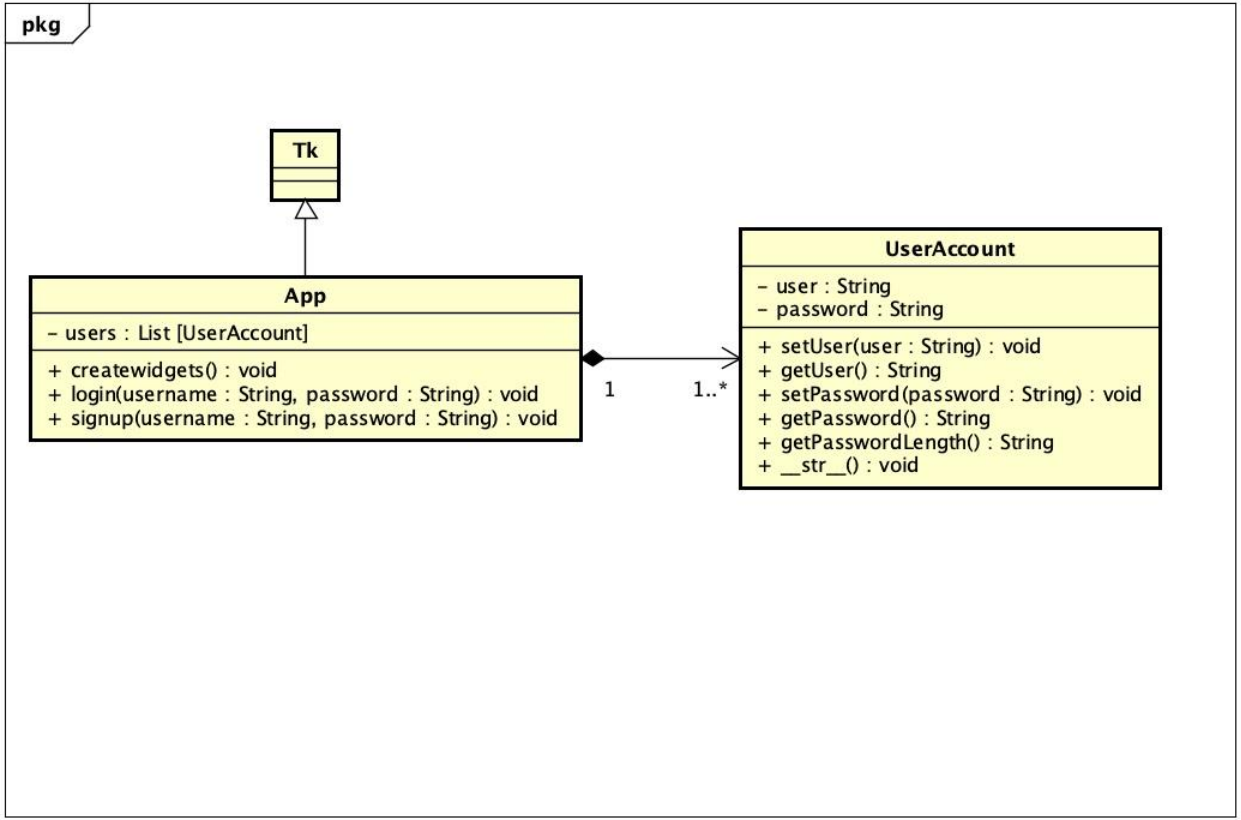
- Signup: Create new user account and keep track of users
- Login: Success message if user/pw exists otherwise error message

Design:

- Introduce UserAccount class
- Introduce signup and login features

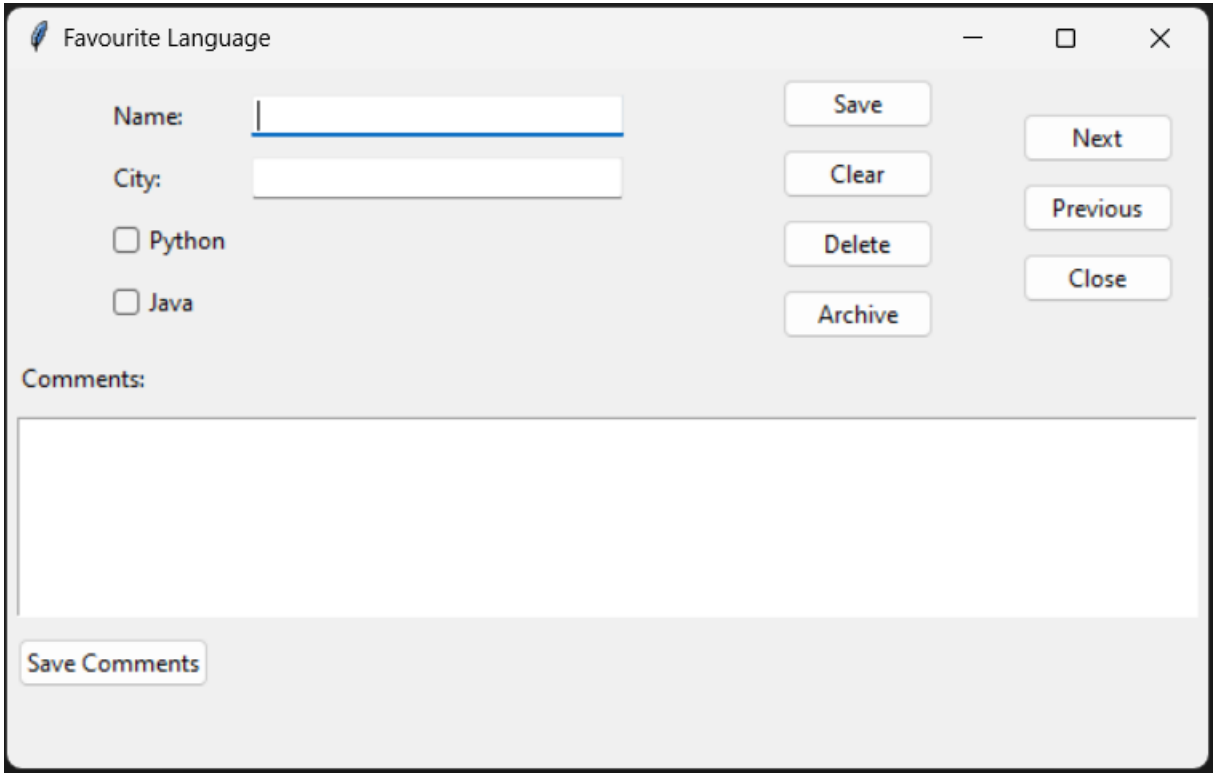
Implementation and Demo loginexamplehomework.py

GUI with Tkinter – Login Example - OOP



Implementation and Demo loginexamplehomework.py

Homework: Can you design a GUI in Tkinter as shown in the image?



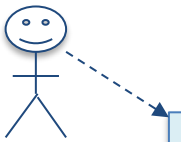
So far we have looked at designing application using Tkinter.

When application grows, its complexity increases!

How should we handle complexity of the application/software?

Model View Controller (MVC)

MVC Design Pattern



User interface
GUI, Tkinter
window, widgets

Business Logic

Data
Data handling
Data validation/integrity

View

Controller

Model

Storage

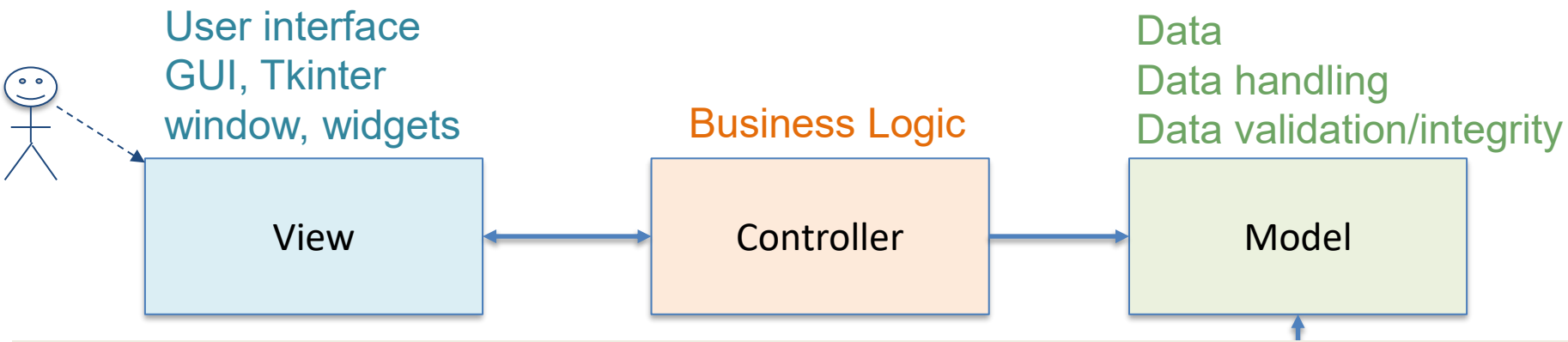
Little logic to display data in the model
Communicates with controller
No direct communication with model

Bridges the gap between model and view

Routes data between view and models

Model must not depend on the view and controller

MVC Design Pattern



Example: if users click the save button on the view, the controller routes the **"save"** action to the model to save the data into a database and notify the view to display a message.

Review this example: https://www.pythontutorial.net/tkinter/tkinter-mvc/

Simplified version

Model

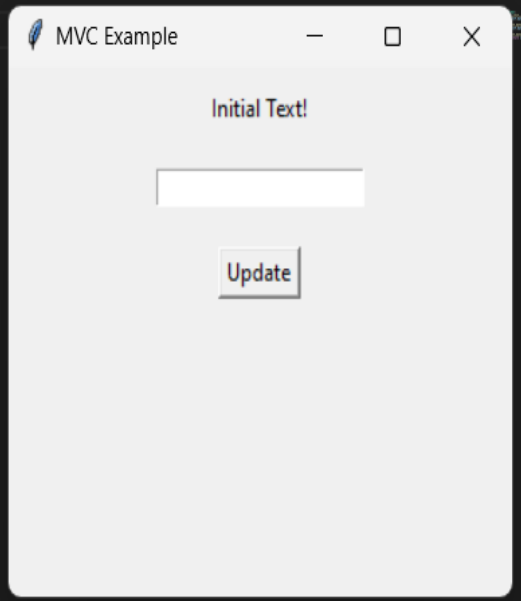
1. Create Model
2. Create View
3. Create Controller
4. Create Main App

App connect Controller with View and Model

main.py | controller.py | view.py | model.py

MVC_Basic_Example > model.py > Model

```python
class Model:
    def __init__(self):
        self.data="Hello, MVC!"

    def set_data(self, data):
        self.data=data

    def get_data(self):
        return self.data
```



MVC_Basic_Example

Example adapted from:
https://www.pythontutorial.net/tkinter/tkinter-mvc/

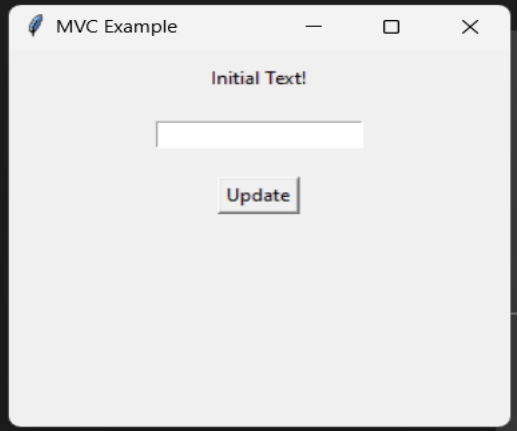
Simplified version

View

main.py | controller.py | view.py

MVC_Basic_Example > view.py > View > update_text

```python
import tkinter as tk

class View:
    def __init__(self, root, controller):
        self.controller=controller
        self.root=root
        self.root.title("MVC Example")
        self.root.geometry('300x300+100+100')

        self.label=tk.Label(root, text="Initial Text!")
        self.label.pack(pady=10)

        self.entry=tk.Entry(root)
        self.entry.pack(pady=10)

        self.button=tk.Button(root, text="Update", command=self.update_text)
        self.button.pack(pady=10)

    def update_text(self):
        user_input=self.entry.get()
        self.controller.update_model(user_input)


    def update_label(self, text):
        self.label.config(text=text)
        self.entry.delete(0, tk.END)
```



MVC_Basic_Example

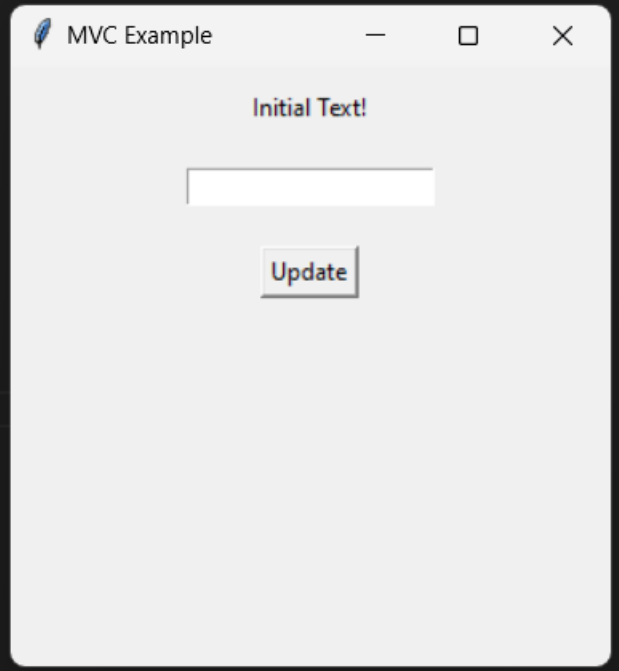
Simplified version

Controller

main.py | controller.py | view.py

MVC_Basic_Example > controller.py > Controller > update_model

```python
from model import Model
from view import View

class Controller:
    def __init__(self, root):
        self.model=Model()
        self.view=View(root, self)

    def update_model(self,data):
        self.model.set_data(data)
        self.view.update_label(self.model.get_data())
```



MVC_Basic_Example

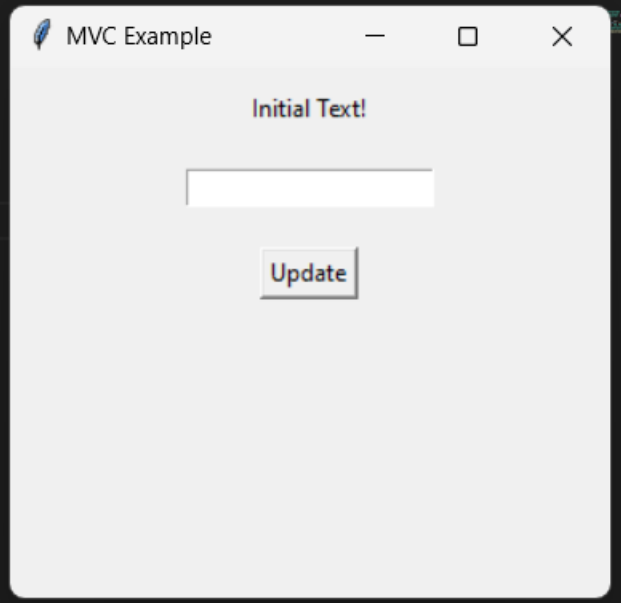
Simplified version

Main App

main.py | controller.py | view.py | model.py

MVC_Basic_Example > main.py > ...

```python
from controller import Controller
import tkinter as tk

if __name__=='__main__':
    root = tk.Tk()
    app=Controller(root)
    root.mainloop()
```



MVC_Basic_Example

MVC Example 2

Model

1. Create Model
2. Create View
3. Create Controller
4. Create Main App

App connect Controller with View and Model

MVC Email Example

Example adopted from https://www.pythontutorial.net/tkinter/tkinter-mvc/

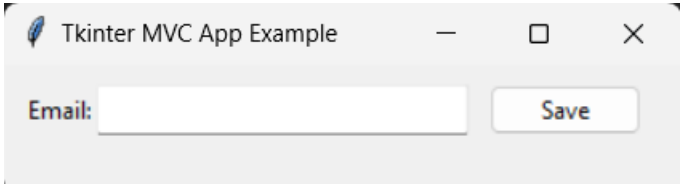
model.py

MVC_Email_Example > model.py > Model > save

```python
import re
#Model class with email property
class Model:
    def __init__(self, email):
        self.__email = email

    def get_email(self):
        return self.__email

    def set_email(self, email):
        if self.validation(email):
            self.__email = email
        else:
            raise ValueError(f'Invalid email address: {email}')

    def validation(self, email):
        #Data validation - email syntax
        pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
        if re.fullmatch(pattern, email):
            return 1 #self.__email = email
        else:
            return 0 #error

    #The save() method writes the email into a simple text file.
    def save(self):
        with open('emails.txt', 'a') as f:
            f.write(self.__email + '\n')
```

MVC Example 2

View



view.py

MVC_Email_Example > view.py > ...

```python
import tkinter as tk
from tkinter import ttk

#view shows a form to input an email
class View(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent)

        # label
        self.label = ttk.Label(self, text='Email:')
        self.label.grid(row=1, column=0)

        # email entry
        self.email_var = tk.StringVar()
        self.email_entry = ttk.Entry(self, textvariable=self.email_var, width=30)
        self.email_entry.grid(row=1, column=1, sticky=tk.NSEW)

        # save button
        self.save_button = ttk.Button(self, text='Save', command=self.save_button_cli
        self.save_button.grid(row=1, column=3, padx=10)

        # message / error message label initalised as empty string
        self.message_label = ttk.Label(self, text='', foreground='green')
        self.message_label.grid(row=2, column=1, sticky=tk.W)

```

MVC Email Example

```python
        # set the controller for View
        self.controller = None

    def set_controller(self, controller):
        self.controller = controller

    def save_button_clicked(self):
        if self.controller:
            self.controller.save(self.email_var.get())

    def show_success(self, message):
        self.message_label['text'] = message
        self.message_label['foreground'] = 'green'
        self.message_label.after(3000, self.hide_message)

        # reset the form
        self.email_entry['foreground'] = 'black'
        self.email_var.set('')

    def hide_message(self):
        self.message_label['text'] = ''

    def show_error(self, message):
        self.message_label['text'] = message
        self.message_label['foreground'] = 'red'
        self.message_label.after(3000, self.hide_message)
        self.email_entry['foreground'] = 'red'
```

MVC Example 2

Controller

MVC Email Example

controller.py

MVC_Email_Example > controller.py > ...

```python
from model import Model
from view import View

#Controller
class Controller:
    def __init__(self, model, view):
        self.model = model
        self.view = view

    def save(self, email):
        try:
            # save the model
            self.model.set_email(email)
            self.model.save()

            # show a success message
            self.view.show_success(f'The email {email} saved!')

        except ValueError as error:
            # show an error message
            self.view.show_error(error)
```

MVC Example 2

Main App

1. Create Model
2. Create View
3. Create Controller
4. Create Main App

App connect Controller with View and Model

MVC Email Example

view.py MVC_LoginExample | main.py MVC_Email_Example | view.py MVC

MVC_Email_Example > main.py > ...

```python
from model import Model
from view import View
from controller import Controller
import tkinter as tk1
from tkinter import ttk

#Application uses model, view and controller
class App(tk1.Tk):
    def __init__(self):
        super().__init__()

        self.title('Tkinter MVC App Example')

        # create a model
        self.model = Model('majid@gmail.com')

        # create a view and place it on the root window
        self.view = View(self)
        self.view.grid(row=0, column=0, padx=10, pady=10)

        # create a controller
        self.controller = Controller(self.model, self.view)

        # set the controller to view
        self.view.set_controller(self.controller)

if __name__ == '__main__':
    app = App()
    app.mainloop()
```

Python Tkinter:

- Official Documentation: https://docs.python.org/3/library/tkinter.html
- Tutorial 1: https://www.pythontutorial.net/tkinter/
- Tutorial 2: https://www.javatpoint.com/python-tkinter
- Tutorial 3: https://www.geeksforgeeks.org/python-gui-tkinter/?ref=lbp

More to do in practical hours

Please make effort to exercise the code in the examples provided. Example code is downloadable from Blackboard Week8 Material.